

As an Executive Director at Nomura managing the Cash Equities Central Risk Book (CRB), my desk is constantly balancing the high-frequency tick data generated by our market-making engines with the longer-term risk horizons required for institutional portfolio rebalancing.

While GARCH models provide a nice parametric framework for daily close-to-close returns, they often fail to capture the "volatility cascade" observed in high-frequency data. Corsi's Heterogeneous Autoregressive (HAR-RV) model is our standard tool for bridging these horizons because it leverages realized variance—the sum of intraday squared returns—making volatility forecasting a simple, robust, and linear regression problem.

## 1. The Mathematical Deep Dive: Step-by-Painstaking-Step

When assessing a Quantitative Researcher, I look for a deep understanding of why the HAR-RV model works despite its mathematical simplicity.

### Step 1: Defining Realized Variance (RV)

We start with intraday returns  $r_{t,j}$  at frequency  $j$  for day  $t$ . The Realized Variance  $RV_t$  is a non-parametric, unbiased estimator of the latent integrated variance. Under the assumption of no jumps and no microstructure noise (which we mitigate with sampling intervals like 5 or 15 minutes):

$$RV_t = \sum_{j=1}^M r_{t,j}^2$$

Where  $M$  is the number of intraday observations.

### Step 2: The Multi-Horizon Decomposition

Corsi's core insight is that market participants operate at different time scales (day traders, short-term hedgers, institutional allocators). Instead of a complex fractionally integrated model (like ARFIMA), we approximate this "long memory" via a simple additive cascade:

$$RV_{t+1} = \beta_0 + \beta_D RV_t + \beta_W RV_t^{(W)} + \beta_M RV_t^{(M)} + \epsilon_{t+1}$$

Where we define the moving averages for volatility as:

- **Daily:**  $RV_t^{(D)} = RV_t$
- **Weekly:**  $RV_t^{(W)} = \frac{1}{5} \sum_{k=0}^4 RV_{t-k}$
- **Monthly:**  $RV_t^{(M)} = \frac{1}{22} \sum_{k=0}^{21} RV_{t-k}$

### Step 3: Why it remains Linear

An ED needs to understand why this is not a non-linear problem. The model is a **linear projection** of future variance onto past realized variance components. Even though the components  $RV_t^{(W)}$  and  $RV_t^{(M)}$  represent different time scales, they are mathematically defined as simple linear combinations of past observables.

Because we are projecting  $RV_{t+1}$  onto a vector  $\mathbf{x}_t = [1, RV_t^{(D)}, RV_t^{(W)}, RV_t^{(M)}]$ , we can estimate the parameter vector  $\beta = [\beta_0, \beta_D, \beta_W, \beta_M]^\top$  using **Ordinary Least Squares (OLS)**:

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{R} \mathbf{V}$$

This provides a closed-form, deterministic solution, avoiding the convergence risks and computational intensity of the iterative Maximum Likelihood Estimation (MLE) required for GARCH models.

## 2. Application of Theory and Code Implementation

For our CRB desk, speed and reliability are paramount. We prefer a clean OLS pipeline to predict  $RV_{t+1}$  across assets, as it integrates seamlessly into our risk management system.

### Production-Grade Python HAR-RV Engine

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

class HAR_RV_Engine:
    """
    Production-grade HAR-RV model for Central Risk Book volatility forecasting.
    Estimates multi-horizon persistence for equity risk assessment.
    """
    def __init__(self):
        self.model_results = None

    def construct_har_features(self, rv_series: pd.Series) -> pd.DataFrame:
        """
        Creates the daily, weekly, and monthly volatility components.
        """
        df = pd.DataFrame({'RV': rv_series})
        df['RV_D'] = df['RV']
        df['RV_W'] = df['RV'].rolling(window=5).mean()
        df['RV_M'] = df['RV'].rolling(window=22).mean()

        # Shift features to predict T+1
        df['Target'] = df['RV'].shift(-1)
        return df.dropna()

    def fit(self, rv_series: pd.Series):
        """
        Estimates the HAR-RV parameters using OLS regression.
        """
        data = self.construct_har_features(rv_series)

        # Prepare Regressors: [Const, RV_D, RV_W, RV_M]
        X = data[['RV_D', 'RV_W', 'RV_M']]
        X = sm.add_constant(X)
        y = data['Target']

        # OLS estimation
        model = sm.OLS(y, X).fit()
        self.model_results = model
```

```

        return model.summary()

def predict(self, recent_rvs: np.array):
    """
    Forecasts next-day volatility based on recent history.
    """
    # recent_rvs should contain at least 22 days of data
    rv_d = recent_rvs[-1]
    rv_w = np.mean(recent_rvs[-5:])
    rv_m = np.mean(recent_rvs[-22:])

    features = np.array([1, rv_d, rv_w, rv_m])
    return np.dot(self.model_results.params, features)

```

### 3. Executive Director's Follow-up Questions

If you presented this to me, I would test your intuition on where this model breaks down.

Follow-Up 1: "The HAR-RV model assumes a constant coefficient for every day in the weekly/monthly window. How do you handle cases where institutional rebalancing (the  $\beta_M$  component) might be triggered by specific calendar events (e.g., month-end), essentially making  $\beta_M$  time-varying?"

- **The Answer:** You are describing a non-stationary regression coefficient. In production, we don't assume  $\beta$  is static. We use a **Rolling Window Regression** or a **Time-Varying Parameter (TVP) HAR** model.
- **The Fix:** We implement a State-Space model (using a Kalman Filter) where the coefficients  $\beta_t$  are treated as latent variables that evolve over time:

$$\beta_{t+1} = \beta_t + \eta_t, \quad \eta_t \sim N(0, Q)$$

This allows the CRB to adaptively adjust how much weight we assign to "monthly" volatility components as we approach month-end, effectively capturing the rebalancing cycle without manually hard-coding calendar triggers.

Follow-Up 2: "In the context of the Cash Equities CRB, realized variance is calculated using intraday prices. How do you handle 'market microstructure noise'—the fact that bid-ask bounces make  $RV$  look larger than the true economic variance at very high frequencies?"

- **The Answer:** Using simple 1-minute  $RV$  would indeed be noisy.
- **The Fix:** We implement **Subsampling** or **Two-Scale Realized Variance (TSRV)**. Instead of using only the 1-minute data, we calculate  $RV$  at multiple frequencies (e.g., 5-min, 10-min, 30-min) and correct for the bias introduced by microstructure noise by subtracting an estimate of the noise variance derived from the highest frequency autocovariances. This ensures our input features into the HAR-RV engine are clean, economic-level volatility estimates.